2.3 Intervalos

Un intervalo (en inglés range) es una notación abreviada para construir listas de valores consecutivos sin escribirlos uno por uno.
Su forma básica es [inicio .. fin], y funciona con cualquier tipo que pertenezca a la clase Enum: enteros, caracteres, booleanos, y también los tipos propios que declares con deriving Enum.
Por debajo, Haskell traduce la notación a funciones de esa clase [a..b] es enumFromTo a b, [a,b..c] es enumFromThenTo a,b,c, [a,b..] es enumFrom a y [a,b...] es enumFromThen a b.

Cuando se escriben los dos primeros elementos, Haskell calcula el paso a partir de la diferencia entre ambos, lo que permite saltos mayores a 1 y también secuencias descendentes. Para contar hacia atrás siempre hay que indicar el paso, porque [10..1] devuelve una lista vacía, ya que el paso por defecto es +1. Si se omite el límite superior se obtiene una lista infinita, que se puede usar sin problema gracias a la evaluación perezosa.

```
main :: IO()
main = do
    print [1..5]        -- [1,2,3,4,5]
    print [2,4..10]     -- [2,4,6,8,10]
    print [10,8..2]     -- [10,8,6,4,2]
    print ['a'..'e']    -- "abcde"
    print (take 3 [1..])  -- [1,2,3]
```

2.4 Operadores

En haskell un operador es una función cuyo nombre está formado por símbolos y que se escribe entre sus dos argumentos (forma infija).
No hay ninguna diferencia esencial entre el operador y una función normal: 3+4 es exactamente lo mismo que (+) 3 4, y cualquier función de dos argumentos puede usarse como operador encerrado su nombre entre acentos graves, como en 10 `div` 3.

Los operadores de uso común se agrupan por su función. Los aritméticos son +, -, * y / (división decimal), junto con div y mod para división y otros enteros, y tres formas de potencia: ^ para exponentes enteros no negativos, ^^ para exponentes enteros que pueden ser negativos y ** para exponentes decimales. Los de comparación son == y /= (distinto no se escribe !=), <, <=, > y >=. Los lógicos son && (y), || (o) y la función not. Para listas se usan: : (agrega un elemento al inicio), ++ (concatena dos listas) y !! (obtiene el elemento en una posición contando desde cero).

2.7. Evaluación Perezosa

Haskell usa evaluación perezosa, también llamada call-by-need: una expresión no se calcula en el momento en que se escribe, si no únicamente cuando su valor hace falta y una vez calculado el resultado se guarda para no repetir el trabajo.

Internamente, cada calculo pendiente se representa con un thunk, una especie de promesa que se resuelve la primera vez que alguien pide el valor.

Las primeras consecuencias es que se pueden estructuras infinitas y consumir solo la parte necesaria.

Una definición como naturales = [1..] no se queda calculando para siempre porque nadie pide la lista completa, y take 5 naturales solo produce 5 elementos.

La segunda consecuencia es que los argumentos no se usan nunca se evalúan, por lo que no pueden provocar errores ni gastar tiempo.

La tercera es que el cortocircuito de &&y|| no necesitan un tratamiento especial en el lenguaje False && undefined devuelve False simplemente porque el segundo argumento no hace falta.

La cuarta es el compartimiento de resultados: si un valor se usa varias veces dentro de una definición, se calcula una sola vez.